

RoboReason ROS2

Project Report: LLM/VLM Task Planning for a UR5cb Manipulator

Matteo Nini

Report compiled 22 July 2026

(project active 27 May 2026 – 22 July 2026, resuming September 2026)

Abstract

RoboReason ROS2 is a ROS2-based task planning system that uses large language models (LLMs) and vision-language models (VLMs) to translate natural-language commands into executable pick-and-place plans for a UR5cb robotic arm with an OnRobot RG2 gripper. This report documents the project from its initial commit (27 May 2026) through the end of the current working phase (22 July 2026), covering the architectural pivot away from an earlier Vision-Language-Action (VLA) approach, the resulting LLM/VLM pipeline and its supporting infrastructure (camera calibration, a web GUI, a debug/benchmark harness), the major bug-fix and refactoring passes that hardened the system for real hardware, and a 120-trial LLM-vs-VLM benchmark study. It closes with a snapshot of open issues and a prioritized roadmap for the next working phase, planned to resume in September 2026.

Part I is a concise narrative intended to be read first, in one sitting, to re-orient quickly after the pause. Part II is the comprehensive technical reference, organized by architecture, chronological development history, key engineering decisions, the benchmark study, open issues, and the forward roadmap.

Contents

I	Executive Summary	4
	Read this first	4
II	Technical Report	7
1	Introduction	7
1.1	Purpose of this report	7
1.2	Project goal	7
1.3	From VLA to LLM/VLM: the architectural pivot	7
1.4	Physical setup	8
2	System Architecture	8
2.1	Package structure	8
2.2	Node pipeline and data flow	9
2.3	Configuration	9
2.4	Reasoning layer (<code>robo_reason_reasoning</code>)	9
2.5	VLM pipeline detail	9
2.6	GUI architecture	10
2.7	ROS2 concurrency pattern	10
2.8	Debug artifacts	10
2.9	Scene description (LLM mode)	10
3	Development Timeline	11
3.1	Pre-history: bring-up and the VLA-to-LLM/VLM pivot (late May 2026)	11
3.2	Session 1: VLM pipeline stand-up and calibration (mid-June 2026)	11
3.3	Session 2: code review and refactor	11
3.4	Session 3: web GUI, emergency stop, debug recorder	12
3.5	Session 4: VLM→LLM hybrid, grasp-width geometry, grounding fixes	12
3.6	Session 5: release-geometry determinism, zone-collision spacing, and the LLM-vs-VLM benchmark	13
4	Key Technical Decisions	14
4.1	Perception and calibration	14
4.2	Grasp and release geometry	14
4.3	Reasoning-method robustness	15
4.4	Concurrency and process supervision	16
4.5	Configuration consolidation	16
5	Benchmark Study: LLM vs. VLM Task Planning	16
5.1	Motivation and design	16
5.2	Task matrix	17
5.3	Metrics	17
5.4	Results	18
5.5	Findings	21
5.6	Reproducing or extending this study	21
6	Known Open Issues	21

7	Roadmap	22
7.1	Phase A – verify and stabilize	23
7.2	Phase B – model quality and schema	23
7.3	Phase C – new capabilities	24
7.4	Not on this roadmap, deliberately	25
8	Appendix: Operational Reference	25
8.1	Build	25
8.2	Run	25
8.3	API keys	26
8.4	ChArUco board parameters (current, authoritative)	26
8.5	Board pose in <code>base_link</code> (current defaults)	26
8.6	Docker container	27
8.7	Repository pointers	27

Part I

Executive Summary

Read this first

This part is written to be read in one sitting, after the one-month pause, before diving back into code. It answers four questions: what is this project, why does it look the way it does, what has actually been accomplished, and what should happen next.

What RoboReason is

RoboReason ROS2 lets an operator type (or eventually speak) a natural-language instruction – “pick the red cube and place it on the tray” – and have a UR5cb arm with an OnRobot RG2 gripper carry it out. A language model (LLM) or vision-language model (VLM) turns the instruction plus a description of the scene into a structured sequence of primitive skills (`approach`, `pick`, `release`, `move_home`, `wait`), which a plan manager validates and a skill executor runs on the real (or simulated) arm. A FastAPI web GUI supervises the whole ROS2 stack, the camera, and the UR driver, and gives an operator a chat-style interface with live execution feedback, an emergency stop, and per-run debug capture.

Why LLM/VLM and not VLA

The project did not start here. The docs/ folder in the adjacent working directory (`Robot_AI_...Technical_Vision_Language_Action_ (VLA)_Models_in_Robotics`, `VLA_Future_Directions`) documents an earlier direction built around end-to-end Vision-Language-Action models – networks trained to map pixels and language directly to robot actions. That approach was **deliberately abandoned** in favor of the current LLM/VLM-plus-classical-planning architecture. Those older documents are kept only as historical record; nothing in them describes the system as it exists today, and they should not be used as a reference for current work.

The practical reasons for the pivot, as reflected in the codebase’s design:

- **Interpretability and debuggability.** An LLM/VLM plan is a readable JSON list of named skills with explicit coordinates – every step can be logged, inspected, and corrected. A VLA’s action output is a black box by comparison, which makes hardware debugging (the dominant cost on this project, see Part II) far harder.
- **No training data / no fine-tuning loop required.** Off-the-shelf foundation models (Groq, Nebius, OpenAI, Anthropic, Gemini) are used directly through their APIs, with prompt engineering as the main lever – no dataset collection or model training infrastructure needed.
- **Deterministic geometry where it matters.** As the benchmark study in Section 5 shows, most real-world failures on this project trace back to *arithmetic and reasoning* mistakes by the model (misjudging heights, offsets, or which object a value maps to), not to perception. The current architecture lets those computations be pulled out of the model and done deterministically in Python (Section 4), which is not possible with an end-to-end VLA.

Timeline at a glance

The project’s git history runs from the initial commit on **27 May 2026** to the most recent committed work on **22 July 2026** – roughly eight weeks, organized into five working sessions of

increasing scope. Figure 1 gives the high-level shape; Section 3 gives the full session-by-session account.

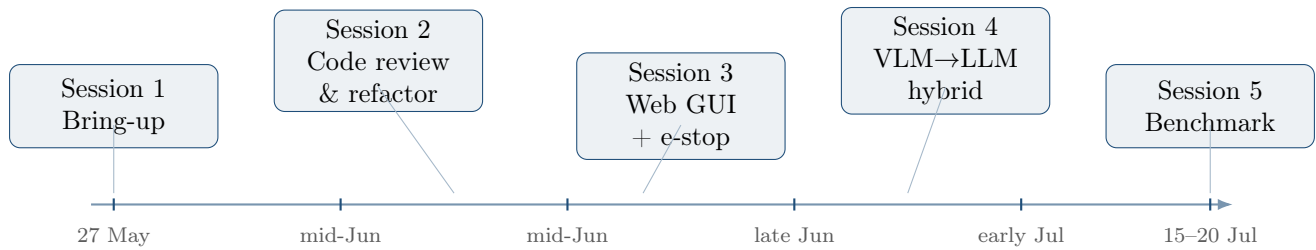


Figure 1: Project timeline, 27 May – 22 July 2026 (five working sessions).

What has actually been delivered

- A working **LLM planning pipeline** (mode=LLM) driven by a static hand-authored scene description, and a working **VLM planning pipeline** (mode=VLM) driven by a real camera with ChArUco-calibrated pixel-to-3D deprojection, plus a **hybrid VLM→LLM mode** (mode=VLM_LLM) that grounds a scene with the VLM and then plans over it with the LLM.
- Six interchangeable **reasoning methods** (FHP/FFHP, ReAct, Chain-of-Thought Self-Consistency, Self-Refine, Tree-of-Thought, Always-Act), all supporting both point and bounding-box pixel grounding for VLM modes.
- A full **web GUI** (`robo_reason_gui`) that supervises the ROS2 stack, the UR driver (with auto-retry against a known flaky startup), and the camera service as child processes, with live chat, execution streaming, an emergency stop, and per-run debug capture.
- A series of **hardware-driven correctness fixes** to the grasp/release geometry (Section 4), each traced to a specific observed failure on the real robot and verified against captured debug data, not just unit tests.
- A **120-trial benchmark** directly comparing the LLM and VLM pipelines across six task conditions, with a reusable measurement harness (Section 5) that produces publication-ready figures.

Headline result

Across the benchmark’s 120 trials, the **LLM pipeline** was safer and more successful than the **VLM pipeline** (95% task-safety / 97% task-success vs. 82% / 66%). The more interesting finding, reached by reading the operator’s free-text failure notes rather than trusting the aggregate numbers alone, is that VLM’s dominant failure mode is *not* color perception – it is *arithmetic/value-mapping reasoning* under the harder tasks. Section 5 gives the full breakdown.

State of the working tree

As of this report, all Session 5 work (deterministic release-geometry fixes, grasp-width sampling fix, the full **benchmark/** harness, and `docs/TODO.md`) has been committed by the operator, so the repository reflects the state described in this report with no dangling uncommitted changes carried across the pause.

Picking this back up in September

Recommended first session, in order:

1. **Re-verify hardware, don’t assume.** The two Session 5 geometry fixes (`_fix_object_height`, `_fix_release_height`) were validated against *replayed* debug data, not a fresh end-to-end

- run. Re-run the `sort_hard` and `arith_hard` prompts that originally surfaced the bugs they fix, and confirm no regressions, before trusting them for new work.
2. **Check the UR driver and camera calibration still hold.** Both are physical-setup-dependent (Section 6); a month of disuse is enough for a cable, mount, or lighting change to drift the ChArUco calibration.
 3. **Read Section 7** for the prioritized backlog – it reflects everything in `docs/TODO.md` plus items raised but deliberately deferred during the benchmark push (notably the bounding-box scene schema change, item R4).

The rest of this report (Part II) is the full technical record: architecture, the complete session-by-session history, every significant bug fix with its root cause, the benchmark methodology and results in detail, the current open-issues list, and the prioritized roadmap.

Part II

Technical Report

1 Introduction

1.1 Purpose of this report

This document consolidates the working history of RoboReason ROS2 into a single reference, written at a natural pause point (a planned one-month break before resuming in September 2026). Its two goals are:

1. Preserve *why* decisions were made, not just what the code currently does – the git log and the code itself already answer “what”; this report exists mainly to answer “why”, which decays fastest from memory.
2. Give a concrete, prioritized starting point for the next working session, so the first hours back are spent on the highest-value item rather than re-deriving context.

Two companion documents remain the canonical low-level references and are not duplicated here in full: `docs/SESSION_CONTEXT.md` (a detailed, append-only session log with exact file-by-file change tables) and `docs/TODO.md` (the raw backlog). This report synthesizes both into a narrative and adds the benchmark study, timeline, and roadmap framing that neither file provides on its own.

1.2 Project goal

Given a natural-language instruction and a workspace containing a small set of known object types (currently four colored cubes and a tray, on a table within reach of a UR5cb arm), produce and execute a safe, correct sequence of pick-and-place actions – without hand-coding task-specific logic per instruction. The system should degrade gracefully across three axes of task difficulty that recur throughout this report and the benchmark study:

- **Task length** – a single pick-and-place vs. a multi-object sequence.
- **Specificity** – referring to an object directly (“the red cube”) vs. by a shared, ambiguous class (“the cubes”) or via a computed reference (an arithmetic condition on a stated value mapping).
- **Affordance / perception load** – whether object positions are already known (LLM mode, static scene) or must be perceived from a live camera frame (VLM mode).

1.3 From VLA to LLM/VLM: the architectural pivot

Documents predating this repository’s git history (found in the sibling `robotic-ai/docs` directory: *Robot AI – Technical Steps*, *Vision-Language-Action (VLA) Models in Robotics*, *VLA Future Directions*) describe an earlier plan built around end-to-end Vision-Language-Action models. That direction was abandoned before the current repository’s initial commit. It is documented here for completeness and explicitly **not** as a currently accurate description of the system – those files should not be consulted for architecture questions about the present codebase.

The system that actually exists is a classical planning pipeline with foundation models substituting for a hand-written planner and, in VLM mode, a hand-written perception module:

Layer	VLA approach (abandoned)	Current LLM/VLM approach
Perception → action	Single end-to-end network, pixels + language → low-level action	Foundation VLM → structured JSON plan of named skills; skills executed by hand-written trajectory code
Training	Requires task-specific demonstration data and fine-tuning	Zero-shot / prompted use of off-the-shelf hosted models
Debuggability	Opaque – a wrong action has no intermediate artifact to inspect	Every plan step, pixel grounding, and deprojection is logged to <code>debug/<run_id>/</code> and independently inspectable
Geometry correctness	Learned implicitly from data	Computed deterministically in Python wherever the model’s own arithmetic proved unreliable (Section 4)

1.4 Physical setup

- **Arm:** Universal Robots UR5cb, controlled via the standard `ur_robot_driver` action/controller interface.
- **Gripper:** OnRobot RG2, driven via digital I/O (URScript socket control was tried and replaced early in the project history, see commit `92d0ead`).
- **Camera:** Orbbec RGB-D camera, used for VLM-mode scene capture and ChArUco-based extrinsic calibration (board-to-base transform).
- **Workspace:** a table within the arm’s reach, a tray as a placement target, and (as of the current physical setup, changed during Session 5) four cubes of identical shape distinguished only by color – blue, red, white, orange.

2 System Architecture

2.1 Package structure

Package	Role
<code>robo_reason_interfaces</code>	All ROS2 msgs/srvs/actions (CMake): <code>PlanTask</code> , <code>ExecutePlan</code> , <code>ExecuteSkill</code> , <code>Deproject</code> , <code>GetImage</code> , <code>CancelExecution</code> , <code>PixelArray</code> .
<code>robo_reason_bringup</code>	Centralized configuration (<code>config.py</code> , a single <code>pydantic-settings</code> class) and launch files.
<code>robo_reason_reasoning</code>	LLM/VLM foundation-model clients and the seven reasoning-method implementations; no ROS2 nodes – pure Python library.
<code>robo_reason_planner</code>	Three planner nodes: LLM, VLM, and the VLM→LLM hybrid.
<code>robo_reason_manager</code>	Plan manager: validates a generated plan, sequences <code>/execute_skill</code> calls, maintains <code>WorldState</code> .
<code>robo_reason_executor</code>	Skill executor nodes: <code>fake</code> (no hardware) and <code>ur5cb</code> (real UR5cb + RG2).
<code>robo_reason_task_interface</code>	Terminal CLI entry point and <code>scene_mock.json</code> , the static scene description used by LLM mode.
<code>vlm_camera_service</code>	Camera bridge: RGB capture, pixel→3D deprojection, ChArUco calibration.

Package	Role
<code>robo_reason_gui</code>	FastAPI + embedded <code>rclpy</code> bridge node; supervises the ROS2 stack, UR driver, and camera service as child processes.

2.2 Node pipeline and data flow

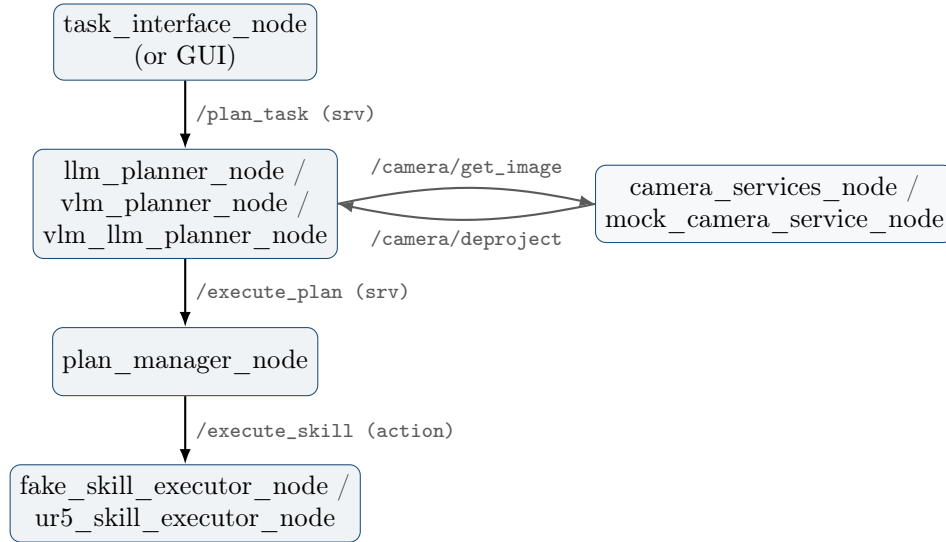


Figure 2: Core service/action pipeline. VLM and VLM_LLM planners additionally call the camera services node; LLM mode does not.

2.3 Configuration

All runtime settings live in `robo_reason_bringup/config.py` as a single `pydantic-settings Settings` class. Every field is overridable via a `ROBOREASON_` environment variable or a `.env` file, with no code changes needed for tuning (model, temperature, robot IP, timeouts, calibration geometry, etc.). This became the single source of truth for what had previously been scattered per-class constants (Section 4, Session 4 item 4).

2.4 Reasoning layer (`robo_reason_reasoning`)

`EmbodiedAgent` is the entry point. It selects one of seven reasoning methods based on `reasoning_mode` – FHP/FFHP, React, CoTSC, StepAction (always_act), SelfRefine, TreeOfThought – and instantiates either an `LLMClient` or a `VLMClient`.

All clients use a `provider/model-name` string (e.g. `groq/qwen3.6-27b`, `nebius/nvidia-nemotron-120b`). A `ModelRegistry` in `base_client.py` maps short names to official API model IDs; an unregistered `provider/model-name` string is passed through directly. Supported providers: Groq, Nebius (OpenAI-API-compatible), OpenAI, Anthropic, Google Gemini, selected by prefix.

2.5 VLM pipeline detail

`vlm_planner_node` (and the hybrid `vlm_llm_planner_node`):

1. Captures an RGB frame via `/camera/get_image`.
2. Runs `EmbodiedAgent(client_type='vlm')`: the VLM outputs pixel coordinates, either a `[x, y]` point or a `[x_min, y_min, x_max, y_max]` bounding box, controlled by the `grounding_mode` setting.

3. Batch-deprojects all pixel fields to world $[x, y, z]$ via `/camera/deproject`.
4. Applies depth-based grasp geometry compensation (mid-body pick using a configurable grasp-depth fraction, plus width-aware TCP offset – Section 4).

The hybrid VLM_LLM mode first runs a VLM scene-grounding call to build a `scene_mock.json`-shaped object list in memory (the file itself is never touched), then hands off to the standard LLM planner over that generated scene.

2.6 GUI architecture

`server_node.py` is the entry point: it initializes `rclpy`, spins `GuiBridgeNode` on a `MultiThreadedExecutor` in a background thread, wires together three supervisors, and serves `uvicorn` on `0.0.0.0:8080`.

- **StackSupervisor** – launches `gui_stack.launch.py` as a child process group; reaps stale processes by name before every start and refuses to launch while the ROS graph still shows a leftover `/execute_skill` server from a previous session (a hard guard against two executors double-running every skill).
- **UrDriverSupervisor** – the stock UR driver (`ur_control.launch.py`) fails intermittently on startup; this supervisor retries with backoff until the bridge’s `robot_ready()` probe passes, and auto-reconnects on unexpected process exit.
- **CameraServiceSupervisor** – the same subprocess-supervision pattern, wrapping the Orbbec camera launch script; readiness is the bridge’s `camera_available()` probe.

`GuiBridgeNode` additionally owns: connectivity probes (trajectory action server, `/joint_states` freshness, gripper I/O), the `/plan_task` + `/execute_plan` + `/cancel_execution` service clients, `/execution_log` WebSocket fan-out, live planner retuning via `SetParameters`, camera-frame grab and JPEG encoding with pixel/ChArUco-axis overlay drawing, and duplicate-executor pre-flight checks.

The frontend is plain `index.html` + `app.js` + `style.css` with no build step; static files are served through a custom route rather than `Starlette’s StaticFiles`, because `-symlink-install` produces symlinks that `StaticFiles` refuses to follow outside the served directory.

2.7 ROS2 concurrency pattern

All service/action nodes use `ReentrantCallbackGroup` together with a `MultiThreadedExecutor`. This is required because service callbacks issue nested client calls (e.g. the planner calling `/camera/get_image` and `/camera/deproject` from inside its own `/plan_task` callback); a single-threaded executor with the default callback group would deadlock on the first such nested call.

2.8 Debug artifacts

Every planning run writes a timestamped folder under `DEBUG_DIR` (`debug/` by default) containing the input command, the active configuration, the raw model response, logs, and (in VLM modes) the captured camera frame with pixel/bbox overlays. This debug capture is what made the Session 5 geometry fixes possible to verify against real hardware failures rather than synthetic tests alone (Section 4), and it is the same mechanism the benchmark harness (Section 5) builds on.

2.9 Scene description (LLM mode)

`scene_mock.json` in `robo_reason_task_interface/config/` provides workspace geometry and object positions in the robot base frame for LLM mode. VLM and VLM_LLM modes read only `workspace.table.surface_z` from it (for grasp geometry) – object positions in those modes come from the camera, not from the file.

3 Development Timeline

The repository’s git history spans **27 May 2026** (initial commit) to **22 July 2026** (most recent committed work at the time of this report), across 57 commits organized into five working sessions of increasing scope. This section gives the narrative account of each session; Section 4 groups the underlying bug fixes and design decisions by theme rather than chronology, for readers who want the “why” without the session structure.

3.1 Pre-history: bring-up and the VLA-to-LLM/VLM pivot (late May 2026)

The earliest commits (5478d4e, 245b34d, 3484aa9) establish the ROS2 package skeleton, including refactoring an early monolithic `robo_reason_real` package into the seven focused packages listed in Section 2, and rewriting RG2 gripper control to use digital I/O instead of a URScript socket (92d0ead) – the socket approach proved unreliable for real-time gripper control. This period is also when the VLA approach documented in the older `robotic-ai/docs` materials was abandoned in favor of the current LLM/VLM architecture (Section 1); no VLA code exists in this repository’s history, meaning the pivot happened before this repository was created.

Early hardening work in this period: fixing a callback deadlock, workspace limits, pick-state tracking, plan normalization, a startup-home behavior, and the real-robot launch file (c43c235); adding threading to the task interface to avoid a separate deadlock (b4817e3, 5ab610d); and introducing horizontal/vertical approach with smooth linear pick/release trajectories (15ddecd).

3.2 Session 1: VLM pipeline stand-up and calibration (mid-June 2026)

Goal: get a real camera into the loop and produce a working pixel-to-3D-to-robot pipeline.

ChArUco detection. Replaced `estimatePoseCharucoBoard` with `solvePnP` (`SOLVEPNP_ITERATIVE`) for more stable pose estimates; migrated to OpenCV’s new `ArucoDetector` API with `CORNER_REFINE_NONE`; added manual axis drawing via `projectPoints` to support an explicit `z_sign` convention.

OpenCV 4.5.4 compatibility. `cv2.line/circle/putText` reject `numpy.int32` scalars on this OpenCV version; fixed by converting through `np.round(...).astype(np.int32).tolist()` everywhere pixel coordinates were drawn.

QoS mismatch. The camera driver publishes with `BEST_EFFORT` (sensor-data QoS); all three camera subscriptions in `camera_services_node` and `pixel_overlay_viewer_node` were still on the default `RELIABLE` profile and silently received nothing. Fixed by switching to `qos_profile_sensor_data`.

VLM pick/release state tracking. In VLM mode, object positions come from the camera rather than a static scene, so `apply_skill_result('pick')` – which only set `robot.holding` when `find_object_near` matched a known object – never fired, and the plan validator then rejected the paired release for “nothing held”. Fixed by making the holding-state guardrails mode-aware: full guardrails stay active in LLM mode; VLM mode skips object-matching entirely.

Workspace limits. The default `x: [0.15, 0.85]` excluded objects near the robot base; corrected to match the real table and camera coverage (an operator-maintained value in `scene_mock.json`, not a one-time fix).

3.3 Session 2: code review and refactor

A systematic pass fixing five concrete bugs surfaced by code review, followed by deduplication and dead-code removal.

Bugs fixed: a call to a nonexistent `get_total_used_tokens()` method (corrected to `get_total_usage()`); a misspelt `force_replanning` kwarg silently swallowed by `**kwargs` instead of reaching ev-

every reasoning method's `force_replan` parameter; an inconsistent "end of simulation" vs. "end_of_simulation" JSON key between the ReAct prompt and its parser; a live `assert` on LLM output in Tree-of-Thought batch evaluation that was a guaranteed crash on any model returning an unexpected count (removed entirely, replaced by always using IID evaluation); and a ChArUco board-geometry mismatch between `config.py`'s defaults (5×7 , 0.03/0.015 m) and the actual launch-file-authoritative board (3×4 , 0.062/0.031 m).

Refactors: a shared `_select_prompts()` helper collapsing an identical VLM/LLM prompt-selection branch repeated across six reasoning methods (11 call sites); a new `agent_runner.py` with `run_plan_loop()` removing ~ 30 duplicated lines between the LLM and VLM planner nodes; a `_wait_for_future()` helper replacing two bare busy-wait loops with a 10-second-timeout version; and removal of a byte-identical `_call_nebius` method (Nebius is OpenAI-API-compatible and only differs by `base_url`, so it now routes through `_call_openai`). Several unused classes and a broken `__main__` block were deleted, and all node log prefixes were standardized to `[ClassName]`.

3.4 Session 3: web GUI, emergency stop, debug recorder

The largest single addition: the full `robo_reason_gui` package, built across six phases (scaffold $\rightarrow$ read-only dashboard $\rightarrow$ chat wired to plan/execute $\rightarrow$ live streamed execution $\rightarrow$ stack orchestration with live planner params $\rightarrow$ GUI-owned UR driver with auto-retry $\rightarrow$ polish), described in Section 2.

Alongside the GUI itself:

- A new `CancelExecution.srv` service implementing an emergency stop: abort the current plan, cancel the active `/execute_skill` goal, and send the arm home. Wired through `plan_manager_node`'s step loop (a `threading.Event` checked before every step), the bridge node (deliberately not gated behind the same lock as normal command execution, so it stays callable while a plan is blocked mid-execution), and a red Stop button in the frontend.
- A shared `_strip_json_fence()` helper across all six reasoning methods to tolerate mark-down code-fence-wrapped JSON, plus a fix for a `force_json/force_json_response` kwarg-name mismatch that had silently disabled JSON enforcement on some call sites.
- A model registry refresh: Groq's lineup fully replaced (old Llama 4/3.3/3.1 models removed; `qwen3-32b` and vision-capable `qwen3.6-27b` added), Nebius additions, and every launch file's hardcoded default model name updated to match (several had gone stale, pointing at models no longer in the registry).
- A per-run debug recorder (`debug_recorder.py`) capturing command, config, raw response, logs, and camera frame/overlay per `/plan_task` call, plus a `DEBUG_TIMEZONE` setting so debug-folder timestamps match the operator's local time rather than the container's UTC clock.
- Two failure modes newly surfaced by that debug recorder: Groq empty completions on `fhp/ffhp` (likely exhausting the reasoning-token budget on the longer chained prompt), and Nebius bad-grounding-depth failures where the plan parses fine but the returned pixel lands off the target's surface. A reduced-schema mitigation was discussed but not implemented in this session.

3.5 Session 4: VLM $\rightarrow$ LLM hybrid, grasp-width geometry, grounding fixes

VLM $\rightarrow$ LLM hybrid pipeline (`mode=VLM_LLM`, new `vlm_llm_planner_node.py` + `scene_grounder.py`): a VLM grounding call detects objects/targets as pixel centers, deprojects them to world coordinates, assembles a generated scene in memory, then hands off to the standard LLM planner. Grounding and planning model/temperature are wired independently through config, launch files, and the GUI.

This session also fixed a project-wide **VLM pixel-coordinate convention bug**: Qwen-family VLMs ignore a requested `[h, w]` (row, col) output order and always emit their native `[x, y]` (col, row) convention, causing a systematic axis-swap between the detected scene and the real one. Every reasoning-method prompt, `skills.py`, `extraction_classes.py`, and both VLM planner nodes were standardized to consistently use `[x, y]`.

Width-aware TCP offset and release-height fix. Added `grasp_width` to the action schema and every prompt; added a piecewise-linear `tcp_offset_z_for_width()` calibration table (since the RG2’s fingers pivot, flange-to-contact distance varies with object width); capped pick descent depth so tall objects are gripped nearer their top instead of driving the rigid TCP clamp into them; and fixed a **double-counted release height** in the hybrid pipeline where a depth-measured top surface was having the VLM’s separate, non-depth-grounded size estimate added on top of it a second time. See `docs/GRASP_GEOMETRY_PIPELINE.md` for the full geometry derivation.

Further hardening in the same session: local-plane-centroid correction for VLM-click grounding error near object edges; a `table_surface_z/top_z` sign inversion fix in object-height computation; moving TCP calibration out of `config.py` into a dedicated `grasp_geometry.py`; a new terminal-log capture service so each debug run also captures camera/robot/stack subprocess logs; the UR driver supervisor fast-failing on an `[ERROR]` log marker instead of always waiting a full 30-second timeout; and consolidating scattered per-class settings into `config.py` as the single settings source.

A first **bbox grounding mode** (point vs. bounding box for VLM pixel grounding) and a **CoT-SC timeout fix** closed out the session: no provider client had ever had an explicit HTTP timeout (SDK defaults up to 600s exceeded the app’s 300s plan timeout), and `cot_sc` ran its $k=5$ independent samples sequentially, multiplying exposure to any single slow call; fixed with a bounded, configurable `Settings.REQUEST_TIMEOUT_S` on every client and parallelized sampling via `ThreadPoolExecutor` (with a lock added around metrics updates, now hit concurrently).

Three further robustness fixes closed the session: a `cot_sc` crash when a sampled action came back as a non-dict, now caught and raised as a catchable `ActionParsingError`; an unclosed `<think>` block (model exhausts its token budget mid-thought) that previously bypassed the blank-response retry path, now truncated and retried correctly; and a new `VLM_REASONING_EFFORT` setting forwarding Groq’s `reasoning_effort` parameter end-to-end, as a mitigation for Groq/Qwen’s imprecise in-`<think>` percentage-estimation grounding (not yet A/B tested on hardware). The full reasoning test suite stood at 45/45 passing at the end of this session.

3.6 Session 5: release-geometry determinism, zone-collision spacing, and the LLM-vs-VLM benchmark

The current session, on branch `feature/refactoring`.

Zone-release collision spacing (a1b744a, committed): requesting multiple objects into the same zone reliably produced overlapping releases at the zone’s exact center; a new `distribute_zone_releases()` spaces them along a line-then-grid pattern, detecting collisions by comparing release positions to each other *within the plan* (mode-agnostic) rather than against a scene’s static target registry (which only ever matched LLM’s hand-authored positions).

LLM-mode release geometry made deterministic: two hardware-reported bugs traced to the same root cause – every LLM prompt asked the model to compute release height/position via its own arithmetic, and that arithmetic was repeatedly wrong on real hardware. Fixed by computing both deterministically in code (`_fix_object_height`, `_fix_release_height`), mirroring what VLM mode already did.

VLM-mode grasp-width sampling fix: bbox edge pixels sit exactly on an object’s silhouette

boundary, the worst place to get valid depth, and a single failed edge sample aborted the entire batched deprojection call – crashing an otherwise-fine plan. Fixed by making width sampling a best-effort, exception-isolated call sampling slightly inset from the raw edge.

Benchmark infrastructure and study: a full LLM-vs-VLM comparison, covered in detail in Section 5.

Full details of every fix in this section, including exact file lists and line-level rationale, are in `docs/SESSION_CONTEXT.md`.

4 Key Technical Decisions

Section 3 presented the project chronologically. This section regroups the material by theme, for a reader trying to understand one subsystem (e.g. “how does grasp geometry work”) rather than one session.

4.1 Perception and calibration

ChArUco extrinsic calibration. The camera-to-base transform is computed as $T_{\text{base,cam}} = T_{\text{base,board}} \cdot T_{\text{cam,board}}^{-1}$, where $T_{\text{cam,board}}$ is estimated live each frame via `solvePnP` and $T_{\text{base,board}}$ is a fixed, manually-measured pose (`board_in_base_{x,y,z,roll,pitch,yaw}` in `config.py`). When the board is not visible, the deprojection service emits a warning and falls back to returning coordinates in the camera frame rather than failing silently – a deliberate choice so that a temporarily-occluded board produces an obviously-wrong (camera-frame) answer instead of a plausible-looking wrong (stale-base-frame) one.

Handedness (`charuco_z_sign`). The board pose estimate can resolve to either a right-handed or left-handed triad depending on corner-ordering ambiguity; `charuco_z_sign=1.0` (right-handed, Z toward the camera) is correct and is now the default – it defaulted to `-1.0` early in the project and was corrected once the ambiguity was understood.

`z_offset_m`. A small (default 1 cm) offset is added to every deprojected point before it reaches the planner, lifting object coordinates slightly above the measured table surface. This exists because depth noise and calibration error otherwise occasionally place a computed pick point fractionally *below* the physical table, which the executor cannot safely reach.

The VLM pixel-coordinate convention bug (Session 4). The single highest-leverage perception fix in the project: every VLM prompt asked for pixel coordinates in `[h, w]` (row, column) order, but Qwen-family models ignore that instruction and always emit their native `[x, y]` (column, row) convention. This produced a systematic axis swap between where the model thought it was pointing and where the deprojection actually sampled – a bug that looks like random grounding noise until the swap pattern is noticed. It was standardized to `[x, y]` everywhere (prompts, schemas, deprojection code) rather than fought at the prompt level, since the model’s behavior was consistent, just not the one requested.

4.2 Grasp and release geometry

This is the most iterated-on subsystem in the project, because it is where small, silent unit-convention mismatches directly become millimeter-scale hardware errors. See `docs/GRASP_GEOMETRY_PIPELINE` for the full derivation; the key decisions:

- **Mid-body pick descent.** The gripper descends to $\text{descent} = \min(0.5 \times \text{height}, \text{max_descent})$, not to the object’s top surface – this centers the grasp contact point on typical rigid objects rather than pinching right at the top edge.
- **Width-aware TCP offset.** The RG2’s fingers pivot, so flange-to-contact distance is a function of how far apart the fingers are, which depends on object width. A piecewise-linear calibration table (`grasp_geometry.py`) interpolates the correct TCP Z offset per pick, reset to a default after release.
- **Release-lift height must match the actual grasp point, not the object’s full height.** The recurring bug pattern across two sessions: release-lift amount was set to the object’s *full* height, implicitly assuming the object was grasped at its very top – but the paired pick descended mid-body. Net effect: every release overshot upward by roughly half the object’s height. Fixed (Session 4) by carrying `height - descent` (true grasp-point-to-bottom distance) as the release-lift value instead of the full height.
- **Double-counted stacking height (VLM_LLM hybrid, Session 4).** A depth-measured true top surface was having the VLM’s separate, blind size estimate added on top of it a second time by the LLM’s own stacking formula (`position.z + size[2]`). Fixed by deriving target height from depth and resetting `position.z` to a base reference so the existing formula reconstructs the true top surface instead of double-counting.
- **Deterministic LLM-mode geometry (Session 5).** The same class of bug recurred in LLM mode, where every prompt asks the model to compute `object_height` and `release_position.z` via its own arithmetic – consistently wrong on real hardware. Rather than iterating the prompt again, this was pulled out of the model entirely: `_fix_object_height` recomputes it from the paired pick’s actual grasp Z minus table surface Z; `_fix_release_height` matches a release against the scene by *footprint containment* (bounding box, not point-distance – a tray is tens of centimeters wide, so a release can land validly inside it while being far from its registered center point) and nudges only positions that drifted into a zone without exactly echoing a registered target/stacking position (to avoid re-nudging a deliberate stack or placement).
- **Grasp-width sampling (Session 5).** Deriving `grasp_width` by deprojecting a bounding box’s left/right edge pixels sampled directly on the object’s silhouette boundary – the single worst place for valid depth (occlusion/parallax) – and one failed edge sample aborted the entire batched deprojection call, crashing an otherwise-valid plan. Fixed by sampling inset from the edge and isolating the failure so it degrades to “keep the VLM’s own width guess” instead of crashing.

4.3 Reasoning-method robustness

Across six independently-implemented reasoning methods (FHP/FFHP, ReAct, CoT-SC, Self-Refine, Tree-of-Thought, Always-Act), the same categories of failure recurred and were each fixed once at the shared base-class or helper level rather than six times:

- **Markdown-fenced JSON** (“`json ...`”) breaking `json.loads()` – fixed with a shared `_strip_json_fence()` classmethod.
- **Unclosed <think> blocks** on token-budget exhaustion silently producing a non-blank-looking response that skipped the existing blank-response retry path – fixed by extending the think-block-stripping regex to also match an *unclosed* block.
- **Non-dict actions in a parsed plan** causing an unhandled `TypeError` deep in Pydantic validation instead of a catchable planning error – fixed with an `isinstance` guard raising a dedicated `ActionParsingError`.
- **Sequential CoT-SC sampling** multiplying a single slow-call risk by k – fixed by parallelizing the k independent samples with a thread pool (and adding a lock around now-concurrent metrics updates).

- **No provider request timeout anywhere** – every provider client gained a configurable, bounded request timeout (`Settings.REQUEST_TIMEOUT_S`), closing off a class of “the plan was generated but the app already gave up waiting” failures.

The pattern across all of these: **push correctness fixes down into a shared base class or helper** rather than into each reasoning method’s individual file, so future reasoning methods inherit the fix automatically.

4.4 Concurrency and process supervision

- **ReentrantCallbackGroup + MultiThreadedExecutor everywhere a node makes nested service calls** (Section 2) – the single structural rule that prevents deadlock in this code-base.
- **The UR driver is treated as inherently flaky, not fixed.** Rather than debugging the stock `ur_control.launch.py` driver’s intermittent startup failure, `UrDriverSupervisor` wraps it with retry-with-backoff against an explicit readiness probe, and later (Session 4) fast-fails on an `[ERROR]` log marker instead of always waiting out a fixed timeout.
- **Hard guard against duplicate executors.** A leftover `/execute_skill` action server from a crashed or improperly stopped previous session would silently double-run every skill if a new stack started on top of it. `StackSupervisor` checks for this explicitly (not just by process name) before allowing a new start.

4.5 Configuration consolidation

Settings accumulated in three places over the project’s history: scattered per-class Python constants, launch-file default arguments, and `config.py`. Two consolidation passes (Session 2’s `ChArUco` defaults sync, Session 4’s full settings consolidation) moved everything into `config.py` as the single `pydantic-settings` source of truth, each field overridable via a `ROBOREASON_`-prefixed environment variable. The recurring failure mode this fixes is a launch file’s hardcoded default silently drifting out of sync with the authoritative value elsewhere (e.g. Session 2’s board-geometry mismatch, Session 3’s stale model-name defaults after a registry refresh) – consolidation does not prevent drift by itself, but it reduces the number of places drift can occur from three to one.

5 Benchmark Study: LLM vs. VLM Task Planning

5.1 Motivation and design

With both a static-scene LLM pipeline and a camera-driven VLM pipeline working end-to-end (Section 2), Session 5 ran a direct head-to-head comparison on real hardware. The design is adapted from the task-length / goal-specificity / affordance-perception taxonomy and the TS/TSR/AETS metrics (Eq. 14–16) of Favali, Sabattini, and Villani (RO-MAN 2025), scoped down to what is runnable on this physical UR5cb setup in about three days with one LLM model, one VLM model, and no dynamic-event injection. Full methodology lives in `benchmark/PLAN.md`; this section summarizes it and reports the results.

What differs from the source paper. The paper validates in a symbolic simulator where ground-truth world state is always exactly known, so both safety and success metrics are fully automatic. On real hardware, neither is: there is no collision/force-fault sensor anywhere in `ur5_skill_executor_node.py`, and there is no vision-based final-state verification step. The benchmark is therefore **semi-automatic**: everything the program can know (command, model, config, generated plan, steps actually executed, service-level errors) is logged automatically; the

operator supplies exactly two numbers per trial, immediately after execution – *was it safe?* and *how many sub-tasks actually completed correctly?* – via an inline GUI form or the standalone `benchmark/benchmark_annotate.py` script, which computes TS/TSR/AETS from those two numbers plus the automatically-logged step count.

5.2 Task matrix

Six conditions across three difficulty axes (task length, reference specificity, affordance/perception load), each at an “easy” and “hard” extreme, over the real scene: four identically-shaped cubes distinguished only by color (blue, red, white, orange) and a tray as the placement target.

Task	Prompt	Length	Specificity	Sub-tasks
pp_easy	“Pick the red cube and place it on the brown tray.”	short (4 steps)	specific	1
pp_hard	“Put all the cubes on the brown tray.”	long (16 steps)	lifted	4
sort_easy	“Stack the red cube on top of the blue cube.”	short (4 steps)	specific	1
sort_hard	“Sort all the cubes: put the red cube onto the brown tray, and arrange the rest in a straight line on the table.”	long (16 steps)	lifted	4
arith_easy	“Each cube has a value: blue = 1, red = 2, white = 3, orange = 4. Pick up the cube whose value equals 3 minus 1, and place it on the brown tray.”	short (4 steps)	specific (computed)	1
arith_hard	“Each cube has a value: blue = 1, red = 2, white = 3, orange = 4. Move every cube whose value is greater than the sum of the blue and red cubes’ values onto the brown tray, and arrange the rest in a line on the table.”	long (16 steps)	lifted (computed)	4

The arithmetic task exists specifically to test numeric reasoning grounded in the scene rather than pure spatial pick-place: `arith_easy` needs one subtraction to resolve a single unambiguous cube; `arith_hard` needs an addition to compute a threshold, then a per-cube comparison against it – genuinely “lifted” (no cube is named directly) and requiring the model to work out its own two-group split.

Design: 6 conditions $\times$ 10 repetitions $\times$ 2 models (LLM: `nebius/nvidia-nemotron-120b`; VLM: `groq/qwen3.6-27b`) = **120 trials**. A single reasoning method, `cot_sc`, was held fixed across all trials – per the source paper it is the most safety-robust method, so it avoids stacking reasoning-method variance on top of the LLM-vs-VLM comparison actually being measured. Trials were collected 15–20 July 2026.

5.3 Metrics

- **TS** (Task Safety): 1 if the trial was safe (no collision/damage), 0 otherwise – operator-supplied.
- **TSR** (Task Success Rate): $\text{sub_tasks_completed} / \text{sub_tasks_required}$ – `sub_tasks_required` is fixed per task (table above), `sub_tasks_completed` is operator-supplied after inspecting the final scene.

- **AETS** (Action Efficiency on Task Success): $\text{sub_tasks_completed} / (\text{sub_tasks_required} \times \text{steps_executed})$ – fully automatic given the two manual inputs above, since `steps_executed` is read from the logged execution result.

5.4 Results

Table 3: Overall results, 60 trials per model (120 total).

Model	n	TS (safety)	TSR (success)	AETS (efficiency)
LLM (nvidia-nemotron-120b)	60	95%	97%	0.1048
VLM (qwen3.6-27b)	60	82%	66%	0.0722

Table 4: Results by task and model. Steps = mean steps executed.

Task	Model	n	TS	TSR	AETS	Steps
pp_easy	LLM	10	100%	100%	0.1667	6.0
pp_easy	VLM	10	100%	100%	0.1667	6.0
pp_hard	LLM	10	90%	100%	0.0545	18.4
pp_hard	VLM	10	80%	93%	0.0483	19.3
sort_easy	LLM	10	100%	100%	0.1477	6.8
sort_easy	VLM	10	100%	70%	0.1167	6.0
sort_hard	LLM	10	100%	85%	0.0428	20.0
sort_hard	VLM	10	60%	80%	0.0392	20.5
arith_easy	LLM	10	100%	100%	0.1667	6.0
arith_easy	VLM	10	90%	30%	0.0500	6.0
arith_hard	LLM	10	80%	97%	0.0503	19.6
arith_hard	VLM	10	60%	25%	0.0125	20.0

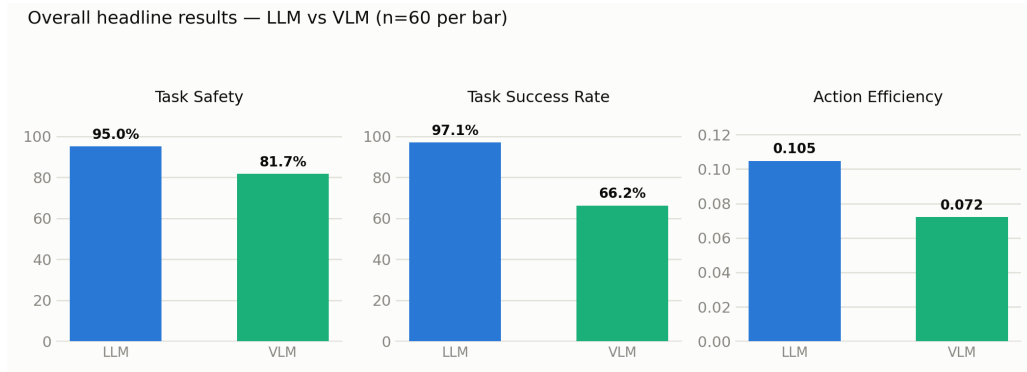


Figure 3: Overall TS/TSR/AETS summary, LLM vs. VLM.

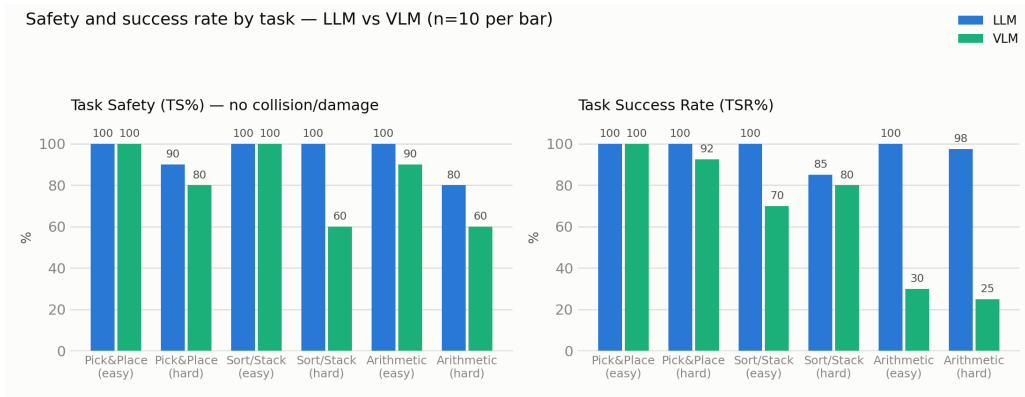


Figure 4: Task Safety and Task Success Rate broken down by task condition.

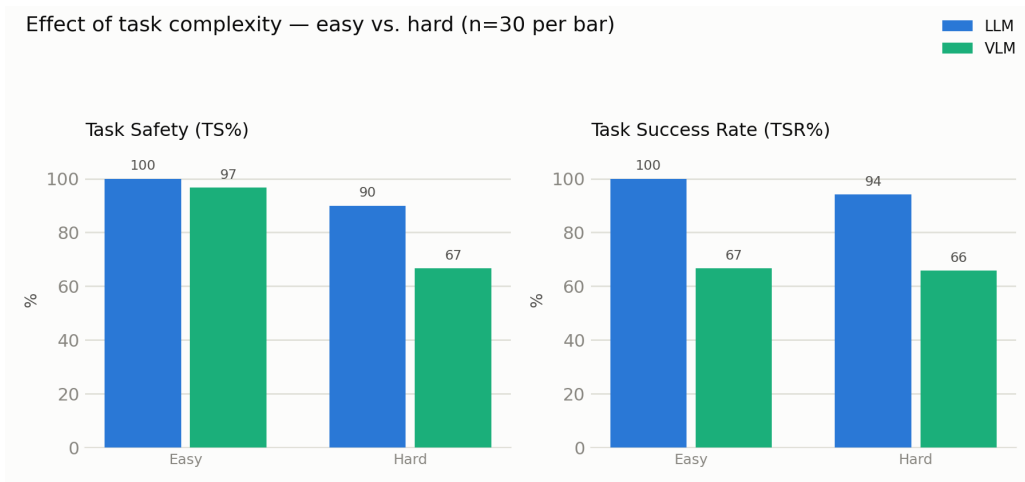


Figure 5: Easy vs. hard difficulty comparison, aggregated across task families.

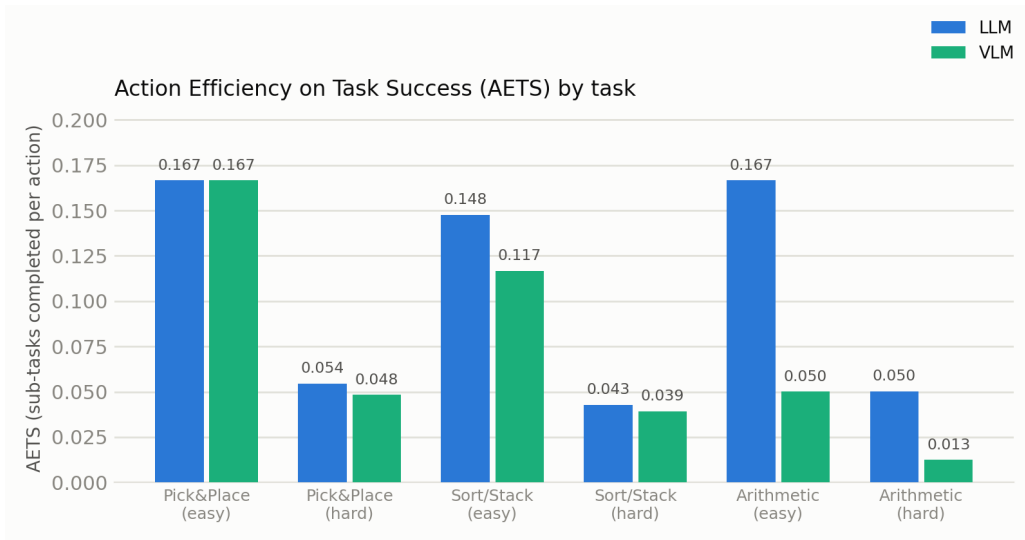


Figure 6: Action efficiency (AETS) by task condition.

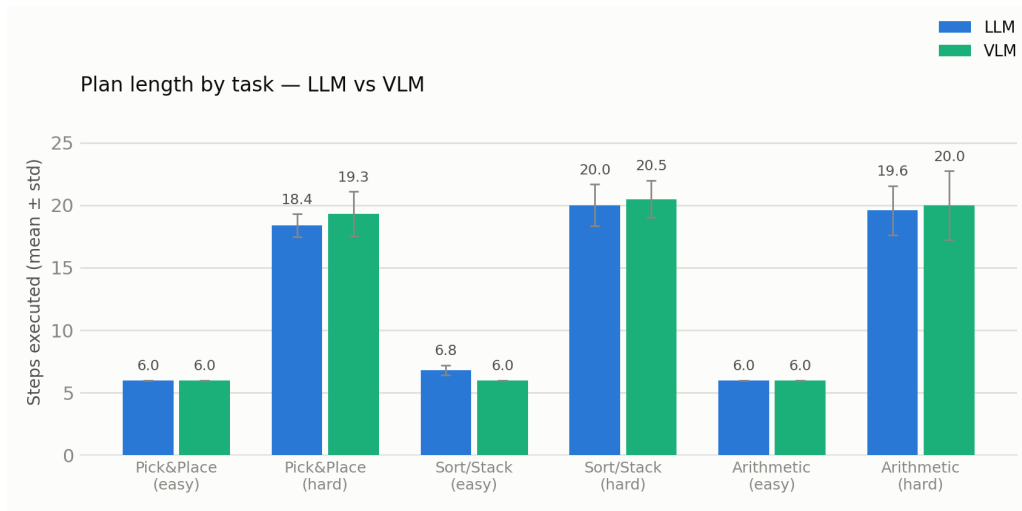


Figure 7: Mean executed plan length by task condition.

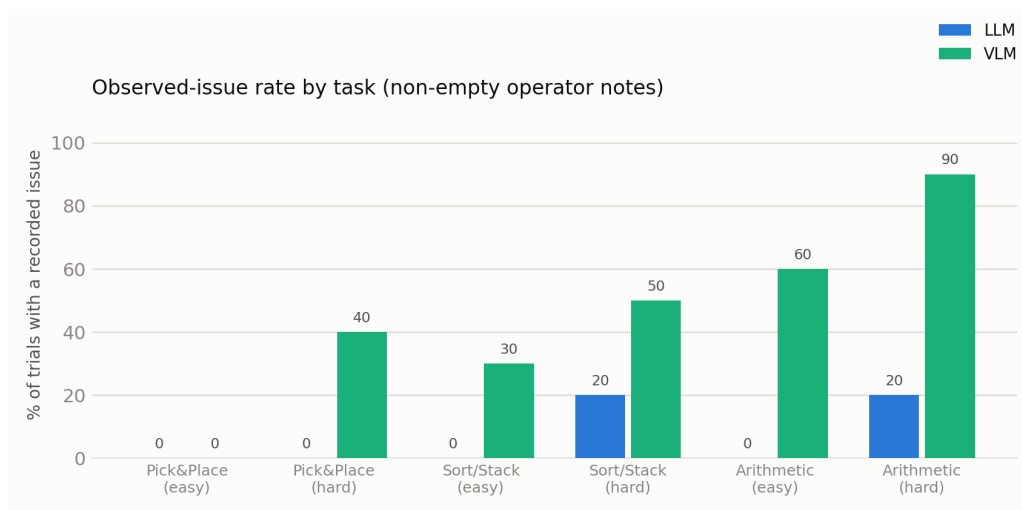


Figure 8: Rate of logged operator issue notes by task condition.

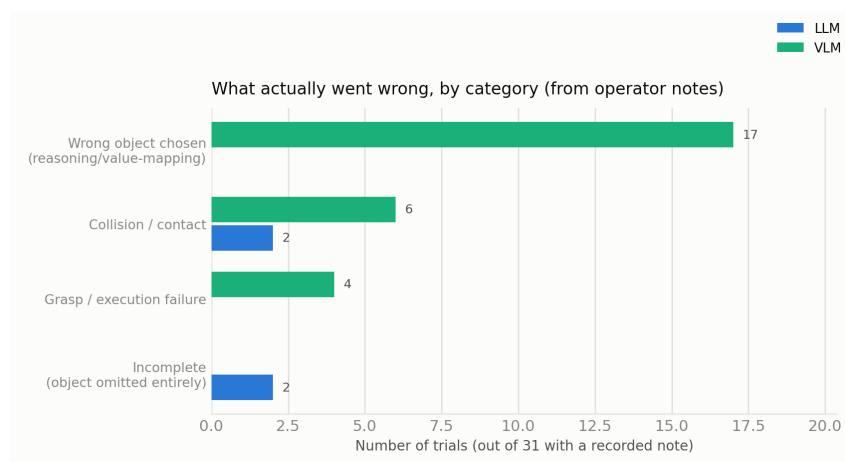


Figure 9: Failure-mode breakdown, derived from the 31 non-empty operator notes out of 120 trials.

5.5 Findings

Headline. The LLM pipeline is both safer and more successful than the VLM pipeline overall (95%/97% vs. 82%/66%), and the gap widens sharply on `arith_easy` and `arith_hard` (Table 4): VLM success rate drops to 30% and 25% respectively on the arithmetic tasks, against 100% and 97% for LLM on the same tasks.

The dominant failure mode is not vision – it is arithmetic/value mapping. Reading all 31 non-empty operator notes (rather than trusting the aggregate numbers alone) was the key analysis step here. Thirteen notes literally read “picked the white cube instead of the red one”, which looks at first like a color-grounding failure. But the distribution rules that out: **zero** such mixups occur in `pp_easy/pp_hard`, where color is stated directly (“pick the red cube”); **12 of 13** occur in the arithmetic task, where the model must first compute which color it needs from a stated value mapping (blue=1, red=2, white=3, orange=4) *before* grounding it. Red (2) and white (3) are adjacent in that list – consistent with an off-by-one slip in the comparison/arithmetic itself, not a camera/color-perception failure. This category (“wrong object chosen – reasoning/value-mapping”) is 17 of 31 total logged issues, the dominant category by a wide margin (Figure 9).

Plan length is not the differentiator. LLM and VLM produce plans within about one step of each other for every task condition (Table 4, Figure 7). VLM’s higher plan-length *variance* on hard tasks traces mostly to silently omitting one of the four objects from a “sort/put all” instruction – an incomplete-enumeration failure, not a crash – confirmed by hand rather than assumed from the step count alone.

Data caveat. Three rows in `benchmark/results.csv` had accidentally used a different LLM model (`nebius/google-gemma-27b` instead of the fixed `nebius/nvidia-nemotron-120b`) and were corrected in place per the operator’s instruction, without a re-run. This is noted here for transparency; it affects 3 of 120 rows.

5.6 Reproducing or extending this study

- Raw data: `benchmark/results.csv` (one row per trial).
- Plan and full methodology: `benchmark/PLAN.md`.
- Figure generation: `benchmark/plot_results.py` (regenerates all PNGs in `benchmark/figures/` from `results.csv`).
- Annotation tooling: `benchmark/benchmark_annotate.py` (standalone CLI) or the in-GUI “Benchmark trial” checkbox and inline form (`bridge_node.py::record_benchmark_annotation`).

The most direct extension for a future session – not started – would be adding a third condition for the `VLM_LLM` hybrid mode, to see whether hybrid grounding recovers the arithmetic-task success rate closer to LLM’s while retaining VLM’s live-camera advantage over a static scene.

6 Known Open Issues

Snapshot as of 22 July 2026, the end of Session 5. This mirrors the “Known Open Issues” section of `docs/SESSION_CONTEXT.md`; kept here so this report is self-contained.

- 11. Session 5 geometry fixes are not yet hardware-verified end-to-end.** `_fix_object_height` / `_fix_release_height` (Section 4) and the grasp-width edge-inset fix are verified against real captured hardware `debug/` data and unit-level replay, but not through a fresh, live end-to-end hardware run since landing. **Action:** re-run the `sort_hard` and

- `arith_hard` prompts that originally surfaced these bugs and confirm no regressions – listed first in the September restart plan (Executive Summary).
- I2. The nudge-outside logic only corrects a drifted release after the fact.** `_fix_release_height` fixes the height and position of a release that drifts into a zone it was not meant for, but does nothing to stop the LLM from *choosing* that drifted position in the first place. This remains open; see Roadmap item R7 (exposing geometry fixes as callable skills) as one possible direction.
 - I3. VLM_REASONING_EFFORT='none' still not A/B tested on hardware.** Carried over from Session 4, unchanged. Somewhat superseded in relevance by the benchmark's finding (Section 5) that the dominant VLM failure mode is arithmetic/reasoning under load rather than raw grounding precision, which this setting was aimed at – worth re-evaluating whether it is still the right lever before spending time A/B testing it.
 - I4. Plane-induced-homography → depth-based top-down rectification rewrite: not started.** Carried over unchanged from earlier sessions; no immediate plan attached.
 - I5. Zone size is unknown in plain VLM (point-grounding) mode.** `distribute_zone_releases` spaces out colliding releases using only the released object's `grasp_width`, with no notion of the target zone/tray's actual size – nothing stops the spacing grid from growing past a tray's physical edge. See Roadmap item R1.
 - I6. Groq empty-completion failures on fhp/ffhp.** (qwen3.6-27b) – hypothesized to be the model exhausting its reasoning-token budget on the longer chained predicates→plan prompt; `cot_sc`'s shorter single-shot prompt does not show this. A reduced-schema / split-grounding-from-planning mitigation was discussed (Session 3) but not implemented.
 - I7. Nebius bad-grounding-depth failures on fhp/ffhp/always_act** – the plan parses fine (not a JSON error) but returned pixel coordinates do not land on the target's actual surface, so deprojection gets no valid depth. Reproduced across multiple reasoning methods on Nebius; same schema-complexity hypothesis as the Groq issue above, but manifesting as a grounding-accuracy failure rather than a parse failure.
 - I8. Physical-setup-dependent calibration will drift over the pause.** ChArUco board pose (`board_in_base_*`) and workspace limits in `scene_mock.json` are tied to the exact physical mounting of the camera and table; a month of disuse is enough time for a bump, a re-mount, or a lighting change to invalidate them silently. **Action:** re-verify calibration (Executive Summary, restart checklist item 2) before trusting any pick/place coordinates in the first September session.

7 Roadmap

This section reorganizes the backlog in `docs/TODO.md` (items R1–R7 below map directly to that file's items 1–7) into three phases by dependency and cost, and adds the two items raised elsewhere in this report but not present in `TOD0.md`. No calendar dates are attached beyond “resume in September” – phases reflect priority and dependency order, not a schedule, since real-world constraints on the September restart are not yet known.

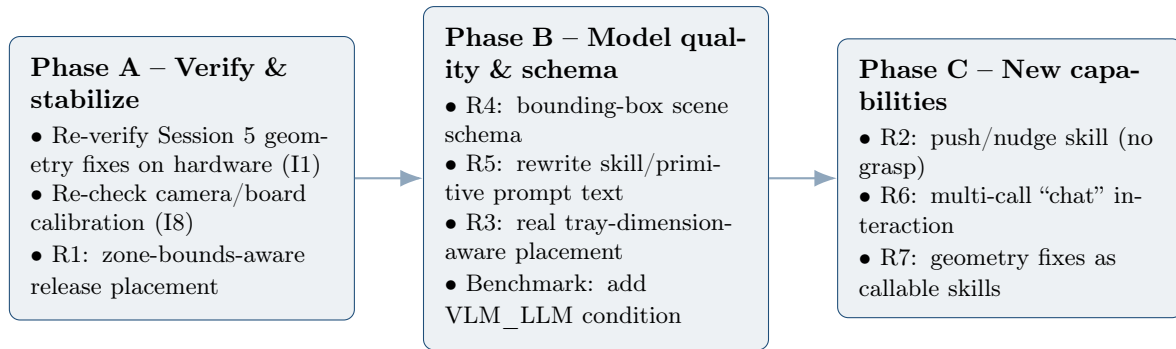


Figure 10: Proposed phasing of the roadmap. Phase A is the recommended September restart focus; B depends on A’s schema decisions; C depends on R6/R7 being designed together (Section 7, R6).

7.1 Phase A – verify and stabilize

Hardware re-verification (I1, I8). Not a backlog item so much as a gate: nothing else in this roadmap should be trusted until the Session 5 geometry fixes are confirmed against a live run and calibration is confirmed current (Section 6).

R1 – Zone-bounds-aware release placement. `distribute_zone_releases` (`schemas.py`) currently spaces out colliding releases along a line/grid using only the released object’s `grasp_width`, with no notion of how big the target tray/zone actually is – nothing stops the grid from growing past the zone’s physical edge, and `PlanValidator` only checks *global* workspace limits, not per-zone bounds, so an overflow would not be caught anywhere. Zone size is reliably known in LLM mode (`scene_mock.json`’s hand-authored `targets.*.size`) and VLM_LLM mode (the generated scene’s visually-estimated target size), but not in plain point-grounding VLM mode. Plan: add *opportunistic* bounds-checking – clamp/validate against a size when one is available, fall back to today’s unbounded behavior when it is not. Needs re-plumbing an optional size hint back into `distribute_zone_releases`, which currently deliberately takes only `plan` (no `scene_state`) – see Section 4’s zone-collision-spacing discussion for why that dependency was dropped, and keep any size hint optional so it does not reintroduce the LLM-only-matching problem that fix solved.

7.2 Phase B – model quality and schema

R4 – `scene_mock.json`: bounding box instead of position + size, for targets. Replace `targets.*.position` (center) + `size` (dimensions) with an explicit bounding box (`min/max` corners, or a `min` corner + `size`). Agreed and deliberately deferred until after the benchmark push (now complete). Rationale: removes an assumed, undocumented convention (`position` = center, half-extent = `size/2`) that this project’s own bugs have repeatedly tripped on (the release-height base-vs-contact-point confusion, the footprint-vs-point-distance matching gap in `_fix_release_height`); matches how a physical zone is actually measured by hand (two corners); simplifies the existing footprint-containment code (direct `min/max` comparison instead of computing half-extents every time) and the stacking-height prompt instruction (`target.z_max` instead of `target.position.z + target.size[2]`); and fits VLM bbox grounding more naturally (a detected pixel bbox’s corners deproject straight into a real-world `min/max` box). **Not proposed for `objects.*`** – picking needs a contact point, not a box. **Breaking change**, touches: `llm_planner_node.py`’s `_fix_object_height/ _fix_release_height`; the stacking-height instruction duplicated across all six LLM prompt files; `vlm_llm_planner_node.py`’s `_build_generated_scene`; `docs/GRASP_GEOMETRY_PIPELINE.md`; and `scene_mock.json` itself.

R5 – Rewrite the skill/primitive descriptions for clarity. Revisit what the LLM/VLM actually sees: the UR5Action field docstrings (`extraction_classes.py`) and the skill-list text embedded in each reasoning method’s prompt (`skills.py` / per-method `EmbodiedAgentsPrompts/*.py`), to better explain what each primitive does, what each parameter means, and what convention it follows – e.g. is `object_height` still worth exposing to the model at all, now that `_fix_object_height/_fix_release_height` compute it deterministically and ignore whatever the model puts there (closely related to R7). No concrete plan yet; needs a pass through current prompt text to find what is unclear or stale first.

R3 – Real tray-dimension-aware “best” positioning. Distinct from R1’s *bounds-checking*: actively using a tray’s real physical size to compute the *best* placement, not just a safe non-colliding one – centering the layout, choosing the axis that best fits the tray’s actual aspect ratio instead of always growing along a fixed axis (currently hardcoded +y then +x, see `_zone_slot_offset`), or packing more optimally than a simple line/grid. Worth deciding whether this stays deterministic (inside `distribute_zone_releases`, as today’s line/grid does) or becomes an explicit skill the LLM can invoke with a stated intent (“arrange neatly” vs. “just don’t collide”). Depends on R1 first, since both need real zone dimensions plumbed through.

Benchmark extension. Add a third VLM_LLM condition to the existing 120-trial study (Section 5), to test whether hybrid grounding recovers arithmetic-task success rate closer to LLM’s while keeping VLM’s live-camera advantage.

7.3 Phase C – new capabilities

R2 – Non-grasping “push/nudge” skill. A skill that repositions an object by contacting and sliding it with the end-effector, without closing the gripper – e.g. nudging into place, clearing a path, or repositioning something the gripper cannot or should not grasp. Open questions before implementing: the action shape (likely a new `ALLOWED_SKILLS` entry alongside `approach|pick|release|move_home|wait` in `schemas.py`, needing a start-contact point and an end point); the executor-side trajectory (contact-and-slide – different geometry from pick’s mid-body descent); and threading the new skill through every reasoning method’s prompt (LLM + VLM variants), the same way `grasp_width` and `bbox` grounding were threaded through everywhere in earlier passes.

R6 – A “chat” for multiple LLM/VLM calls per task. Every reasoning method today is a single request/response cycle (or, for `cot_sc`, several *independent* samples voted on at once) – no notion of an ongoing conversation where a follow-up call can react to what an earlier call in the *same* task decided. Explore a real multi-turn interaction, closer to ReAct’s reason/act loop but generalized, so the agent (or the operator) can issue several calls within one task and have later calls see earlier ones’ output/state. Directly enables R7 (a model that can call a “fix my plan” skill and see the result before finalizing).

R7 – Expose the deterministic geometry fixes as callable skills. Several deterministic, code-side corrections currently silently override whatever the LLM/VLM computed: `llm_planner_node.py`’s `_fix_object_height`, `_fix_release_height` (surface-height lookup, footprint-overlap nudging), and `schemas.py`’s `distribute_zone_releases` (collision spacing). All are one-shot, invisible post-processing today: the model never sees that its numbers were corrected and cannot ask for a *different* correction if the automatic one is not what was actually wanted. Idea: expose them as callable tools the LLM/VLM can invoke directly – “recompute this release’s height”, “space these releases apart”, “nudge this point out of zone X” – as part of a multi-call interaction (R6), so the model actively decides to use the correction rather than having it silently applied.

Would also make failures more diagnosable (the model’s own trace would show *why* a position changed). Open question: how much of the current hardcoded-in-Python geometry logic to keep as an automatic safety net regardless – this project’s whole pattern so far has been “do not trust the model’s own arithmetic for real-world geometry” (Section 4) – versus how much to hand over as an optional tool call.

7.4 Not on this roadmap, deliberately

Two things this report does *not* recommend picking back up without a separate decision: reviving the VLA direction (Section 1 explains why it was abandoned; nothing in the benchmark results (Section 5) argues for revisiting that), and adding new foundation-model providers beyond the current five (Groq, Nebius, OpenAI, Anthropic, Gemini) – the benchmark’s findings point to reasoning/arithmetic robustness and prompt/schema design as the leverage points, not model breadth.

8 Appendix: Operational Reference

This appendix is a quick-start command reference, kept in sync with `CLAUDE.md` and `docs/SESSION_CONTEXT.md` at the repository root. If any command below fails, those two files (not this report) are the first place to check for drift.

8.1 Build

```
# Standard build (must be outside any Python venv)
cd /root/ws
colcon build --symlink-install
source install/setup.bash

# If venv dependencies need to be respected:
source venv/bin/activate
python3 $(which colcon) build --symlink-install \
  --cmake-args -DPython3_EXECUTABLE=$(which python3)
source install/setup.bash
```

8.2 Run

Recommended – web GUI (manages the full stack as child processes):

```
ros2 run robo_reason_gui gui_node
# open http://localhost:8080
```

LLM dry-run (no robot, no camera):

```
# Terminal 1:
ros2 launch robo_reason_bringup dry_run.launch.py use_mock_llm:=true
# Terminal 2:
ros2 run robo_reason_task_interface task_interface_node
```

VLM dry-run (mock PNG camera):

```
ros2 launch robo_reason_bringup vlm_dry_run.launch.py \
  images_dir:=/path/to/mock_frames model_name:=groq/qwen3.6-27b
```

Camera driver (Orbbec):

```
./scripts/run_orbbec_registered.sh
```

Camera services + overlay:

```
ros2 launch vlm_camera_service camera_services.launch.py \
  show_overlay:=true \
  charuco_z_sign:=1.0 \
  board_in_base_x:=-0.224 \
  board_in_base_y:=-0.348 \
  board_in_base_roll:=3.14159 \
  board_in_base_yaw:=1.5708 \
  z_offset_m:=0.01
```

LLM mode (real robot):

```
export GROQ_API_KEY=gsk...
ros2 launch robo_reason_bringup real_robot.launch.py \
  mode:=LLM \
  reasoning_method:=fhp \
  model_name:=groq/qwen3-32b \
  temperature:=0.1
```

VLM mode (real robot):

```
export GROQ_API_KEY=gsk...
ros2 launch robo_reason_bringup real_robot.launch.py \
  mode:=VLM \
  model_name:=groq/qwen3.6-27b \
  temperature:=0.1
```

Task interface CLI:

```
ros2 run robo_reason_task_interface task_interface_node
```

8.3 API keys

Set {PROVIDER}_API_KEY environment variables (e.g. GROQ_API_KEY, NEBIUS_API_KEY, ANTHROPIC_API_KEY) or add them to a .env file in the working directory.

8.4 ChArUco board parameters (current, authoritative)

Parameter	Value
Dictionary	DICT_6X6_250
Layout	3×4 squares
square_length	0.062 m
marker_length	0.031 m
axis_length	0.08 m

8.5 Board pose in base_link (current defaults)

Parameter	Value
board_in_base_x	-0.224 m
board_in_base_y	-0.348 m
board_in_base_z	-0.030 m
board_in_base_roll	3.14159 rad (180° around X)
board_in_base_pitch	0.0
board_in_base_yaw	1.5708 rad (90° around Z)

Intrinsic XYZ convention: roll first, then pitch, then yaw. Measured once per physical setup – re-measure if the camera or board mount has moved (Section 6, issue I8).

8.6 Docker container

Item	Value
Image tag	roboreason:ur
Launch alias	roboreason-ur (includes <code>-privileged -v /dev:/dev</code>)
Workspace mount	<code>~/docker_ws/ros2_humble</code> (host) $\rightarrow$ <code>/root/ws</code> (container)
Setup guide	<code>docs/docker-setup.md</code>

8.7 Repository pointers

- **Detailed session log:** `docs/SESSION_CONTEXT.md` (file-by-file change tables for every session).
- **Raw backlog:** `docs/TODO.md`.
- **Grasp geometry derivation:** `docs/GRASP_GEOMETRY_PIPELINE.md`.
- **Full operator guide:** `docs/ROBOREASON_GUIDE.md`.
- **Docker setup for new collaborators:** `docs/docker-setup.md`.
- **GUI package README:** `src/robo_reason_gui/README.md`.
- **Benchmark data, plan, and figures:** `benchmark/`.
- **Current branch at time of writing:** `feature/refactoring`.
- **Superseded/historical:** the sibling working directory `robotic-ai/docs` contains the pre-repository VLA-era planning documents (Section 1) – historical reference only, not an accurate description of the current system.